About this PDF

Built only from: **AL3501 Unit V: Discourse Analysis and Lexical Resources (Verified Academic Notes)** — local:AL3501/NLP AL3501_UnitV_VerifiedNotes.pdf

Licence: CC BY-SA 4.0. Accessed 2026-10-05.

Covers 9 of 9 syllabus sub-topics of this unit.

Discourse segmentation

Definition

Discourse segmentation is the process of locating the boundaries between Elementary Discourse Units (EDUs) in a tokenised text T = (t1,…,tn). The output is a sequence of indices 0 = s0 < s1 < … < sk = n such that each interval (s_{i-1}+1 … s_i) forms an EDU, and the set {s1,…,s_{k-1}} are called EDU boundaries.

Explanation

Accurate segmentation provides the backbone for coherence modelling and coreference resolution, which are essential for building intelligent language systems. Boundaries are signalled by punctuation, discourse connectives, or syntactic cues such as a subject-verb pattern. The rule-based approach places EDU boundaries at punctuation marks (periods, commas, semicolons) and at discourse connectives; this method is fast and language-independent but may miss subtle clause splits. Statistical or neural models learn to label each token as beginning (BEDU) or inside (IEDU) an EDU using features like POS tags and connective cues, allowing prediction of boundaries even without explicit punctuation.

Example

In the sentence “She read the book, and she liked it”, “She read the book” and “she liked it” are two EDUs, with the comma followed by the connective “and” marking the boundary. Another illustration: “She laughed, she cried, she sang.” each comma creates a new EDU.

Algorithm / Steps

1. Initialise an empty list EDU_list and an empty buffer curr_EDU.
2. Scan the token list from left to right, appending each token t to curr_EDU.
3. If t matches a boundary cue (punctuation, discourse connective, or optional syntactic marker), copy curr_EDU to EDU_list and reset curr_EDU to empty.
4. After the loop, if curr_EDU still contains tokens, append it as the last EDU to EDU_list.
5. Return EDU_list as the final segmentation result.

Key points

- An EDU is the smallest span that can serve as a single argument of a discourse relation.
- Boundary cues are punctuation or discourse connectives; a new EDU starts after any cue.

- Errors in boundary detection can corrupt the entire discourse tree, making later correction difficult.
- Segmentation feeds tokenisation and POS tagging and supplies boundaries for coreference resolution and lexical resources.

Coherence

Definition

Coherence refers to the logical link (such as Cause, Contrast, Elaboration, etc.) that connects two adjacent Elementary Discourse Units (EDUs) after the text has been segmented. It models how the meaning of one EDU relates to the next, enabling the construction of a coherent discourse structure.

Explanation

After discourse segmentation identifies EDU boundaries, a parser assigns a coherence relation to each pair of adjacent EDUs. These relations capture the inferential role that one EDU plays for the hearer, guiding downstream tasks like summarisation and dialogue management. Accurate segmentation is therefore crucial because it provides the backbone for coherence modelling; any error in boundary detection can lead to incorrect relation assignment and degrade system performance.

Example

In the sentence pair “It rained” and “the streets were wet”, the coherence relation is a **Cause** relation, indicating that the first EDU explains the reason for the second.

Key points

- Coherence relations are logical links (Cause, Contrast, Elaboration, etc.) between adjacent EDUs.
- They are assigned after discourse segmentation and are essential for building discourse trees.
- Accurate EDU boundaries are critical because errors propagate to coherence modelling, affecting downstream applications.

Discourse Reference Phenomena

Explanation

Discourse reference phenomena rely on the boundaries identified during discourse segmentation; these boundaries are used by coreference resolution to link referring expressions across the text.

Key points

- Coreference resolution uses EDU boundaries produced by discourse segmentation.
- Accurate segmentation is crucial because incorrect boundaries can lead to erroneous reference linking and degrade downstream NLP tasks.

AcademicsGen — source: VERIFIED_NOTES — CC BY-SA 4.0

Anaphora Resolution using Hobbs and Centering Algorithm

Definition

Anaphora resolution is the process of identifying the antecedent of an anaphoric expression, such as a pronoun, within a discourse. It is a crucial task in natural language processing because it enables the correct interpretation of references and contributes to overall text understanding.

Explanation

Two major approaches are discussed: Hobbs' algorithm and the Centering Theory approach. Hobbs' algorithm follows a systematic search strategy through the syntactic parse tree to locate a suitable antecedent, while Centering Theory focuses on the discourse-level coherence and the salience of entities to determine likely antecedents. The two methods differ in their underlying principles—syntactic traversal versus discourse salience.

Key points

- Anaphora resolution links pronouns and other referring expressions to their correct antecedents.
- Hobbs' algorithm uses a tree-based search procedure, whereas Centering Theory relies on discourse coherence and entity prominence.
- Understanding both approaches helps in selecting appropriate techniques for different NLP applications.

Coreference Resolution

Key points

- The boundaries identified during discourse segmentation are used by coreference resolution modules to link referring expressions across the text

Resources: Porter Stemmer and Lemmatizer

Definition

The Porter stemmer is a lexical tool used for word-level normalization within discourse processing pipelines, while a lemmatizer maps inflected word forms to their canonical lemmas through morphological analysis and lexical lookup.

Explanation

The Porter stemmer reduces words to a base stem by stripping common suffixes according to a series of heuristic rules; this helps downstream modules treat different surface forms as the same lexical item. A lemmatizer analyses suffixes and consults a dictionary or exception list to remove inflectional endings, producing the base form of a word; it operates token by token and combines rule-based suffix stripping with lexical lookup.

Example

Applying the Porter stemmer to the word “running” yields the stem “run”.

Key points

AcademicsGen — source: VERIFIED_NOTES — CC BY-SA 4.0

- Both tools are employed after discourse segmentation to normalize words before further analysis.
- The Porter stemmer uses heuristic suffix-stripping rules, whereas a lemmatizer relies on morphological analysis and dictionary lookup.
- Normalization enables modules such as parsers and semantic role labelers to treat variants like “run”, “runs”, and “ran” as the same concept.

Penn Treebank and Brill's Tagger

Definition

A **Penn Treebank** is a 45-tag part-of-speech tagset that assigns grammatical categories such as NN, VB, JJ to each token. A **Brill's Tagger** is a rule-based part-of-speech tagger that starts with a simple lexical tagger and then refines tags using transformation rules.

Explanation

The Penn Treebank tagset provides a standard way to represent grammatical categories, enabling models to learn from uniform data. Brill's Tagger improves tagging accuracy without heavy statistical training by iteratively applying hand-crafted or automatically learned transformation rules to correct an initial tag sequence. The learning process is error-driven: rules are generated to fix the most frequent mismatches between the tagger’s output and a gold standard corpus.

Example

Sentence: “The dogs bark” Initial tags (lexicon): The/DT dogs/NNS bark/VB Rule generated: “change VB to VBP if previous tag is NNS” After applying the rule: The/DT dogs/NNS bark/VBP – the verb tag is now consistent with the plural subject.

Algorithm / Steps

1. Assign each token an initial tag using a lexical lookup table; unknown words receive the most frequent tag for their suffix.
2. Compare the initial tag sequence with a gold standard annotated corpus to find mismatched tokens and count error types.
3. Generate candidate transformation rules (e.g., “change tag A to B if previous tag is C”) that would correct the identified errors.
4. Rank the candidate rules by the net reduction in tagging errors; select the highest scoring rule and add it to the rule list.
5. Apply the selected rule to all sentences, update the tags, and repeat Steps 2–4 until no rule yields a significant error reduction.

Key points

- Penn Treebank tagset contains 45 tags and is widely accepted for POS tagging.
- Brill's Tagger uses an initial lexical tagger followed by ordered transformation rules.
- The tagger’s learning objective is to maximise error reduction, E = E_before − E_after for each rule.

- Brill's Tagger is useful in limited-resource settings and produces interpretable rules.
- Applications include search indexing, grammar checking, and information extraction.

WordNet, PropBank, FrameNet

Definition

WordNet is a lexical database of English organized into synonym sets (synsets) linked by semantic relations. PropBank is an annotation layer on the Penn Treebank that marks verb predicates with generic protoroles (ARG0 ARG5) and optional modifiers (ARGM*). FrameNet is a frame-based lexical semantic resource where each frame defines a conceptual situation and a set of named frame elements (FEs) that act as role labels.

Explanation

WordNet groups synonyms into synsets and connects them with relations such as hypernymy, meronymy, and antonymy, forming a directed acyclic graph that supports selectional restriction checks during role labeling. PropBank attaches to each verb sense a small set of generic roles (ARG0, ARG1, etc.) that describe the core participants of the event. FrameNet organizes words into frames that describe a situation and defines named frame elements (e.g., Buyer, Seller) that are stable across many lexical units. These resources provide the semantic knowledge that plain POS tags or parse trees lack, enabling systems to know who did what to whom, to select appropriate word senses, and to enforce selectional restrictions.

Example

1. Predicate detection: identify verb “approved” as predicate p.
2. Parse tree: (S (NP (DT The) (NN board)) (VP (VBD approved) (NP (DT the) (NN budget)) (ADVP (NN yesterday))).
3. Candidate constituents under p: c1 = “The board” (NP before verb), c2 = “the budget” (NP after verb), c3 = “yesterday” (ADVP).
4. Feature vectors (sample):
   - c1: POS=NP, position=before verb, WordNet hypernym=organization predicts ARG0.
   - c2: POS=NP, position=after verb, WordNet hypernym=financialdocument predicts ARG1.
   - c3: POS=ADVP, temporal cue word predicts ARGM TMP.
5. Classification results: c1 ARG0, c2 ARG1, c3 ARGM TMP.
6. Consistency check: no duplicate ARG0, spans do not overlap, ordering satisfied. Final SRL output: ARG0 = The board, Predicate = approved, ARG1 = the budget, ARGM TMP = yesterday.

Algorithm / Steps

1. **Predicate detection** – Scan the token list and POS tags to locate all verb tokens (or noun tokens for NomBank).
2. **Syntactic parsing** – Generate a constituency parse tree for the sentence; this tree supplies structural paths for feature extraction.
3. **Feature extraction** – For each (predicate, constituent) pair, build a feature vector containing lexical, syntactic, positional, and WordNet based features.
4. **Classification** – Apply a trained multiclass classifier (e.g., MaxEnt or SVM) to predict a role label from the set {ARG0, ARG1, …, ARGM*, NONE}.
5. **Global consistency check** – Enforce constraints such as no duplicate ARG0 for the same predicate, non-overlapping spans, and preferred ordering of arguments.
6. **Output** – Produce the final SRL assignment for each predicate.

Key points

- WordNet is a lexical database of synsets linked by semantic relations.
- PropBank annotates verb senses with generic roles ARG0–ARG5 and optional modifiers.
- FrameNet groups words into frames and defines named frame elements.
- SRL pipelines use predicate detection, parsing, feature extraction, classification, and consistency checks.
- Selectional restrictions link roles to WordNet synsets, improving accuracy.

Brown Corpus and British National Corpus (BNC)

Definition

A corpus is a curated collection of real sentences. The Brown Corpus is a small, genre-balanced collection of about one million words written in American English from the 1960s. The British National Corpus (BNC) is a much larger, 10 million-word library that contains both written and spoken British English. Both corpora act as reference books for researchers to see how words are actually used.

Explanation

The Brown Corpus contains ~1 M tokens and 38 k types, while the BNC holds about 10 M tokens and many more types. Brown is deliberately balanced across 15 genres (news, fiction, essays, etc.), which reduces domain bias. BNC includes additional spoken genres such as radio transcripts and conversations, and supplies lemmas and speaker information for spoken data. Both corpora provide POS tags for every token; BNC also supplies lemmas and speaker information for spoken data. No syntactic trees are included, unlike the Penn Treebank.

AcademicsGen — source: VERIFIED_NOTES — CC BY-SA 4.0

Example

A news article from Brown and a telephone conversation from BNC illustrate the same word “government” used in different contexts. The sentence “President Kennedy today pushed aside other White House business …” is a classic Brown example. A BNC transcript of a BBC radio interview provides realistic dialogue for ASR training.

Algorithm / Steps

1. Import the NLTK corpus module and load the desired corpus into memory.
2. If a genre filter is required, select the subset of texts that belong to that genre.
3. Iterate over the token list and count occurrences of the query word or tag.
4. Store counts in a dictionary or use NLTK’s FreqDist to obtain a frequency distribution.
5. Return the final list or distribution; optionally sort by descending frequency.
6. loads the brown corpus.
7. is omitted because we use the whole corpus.
8. creates a lower-cased generator of all tokens.
9. builds a FreqDist object that counts each word.
10. prints the count for “the”, which is 6235 occurrences.

Key points

- Brown Corpus: ~1 M tokens, 38 k types, 15 balanced genres, POS tags.
- BNC: ~10 M tokens, includes spoken data, provides lemmas and speaker info.
- Both corpora are freely downloadable and integrated into popular NLP libraries (NLTK, spaCy).
- Larger size of BNC yields richer frequency information and better coverage of rare words.
- Genre balance in Brown reduces domain bias, while BNC’s spoken genres add realism for speech-related models.

Source and licence

This PDF is adapted from **AL3501 Unit V: Discourse Analysis and Lexical Resources (Verified Academic Notes)** (local:AL3501/NLP AL3501_UnitV_VerifiedNotes.pdf), accessed 2026-10-05.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).